MTH251

Timed Online Assignment – January Semester 2021

Data Structures and Algorithms I

Monday, 17 May 2021 **4:00 pm – 6:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **EIGHT (8)** questions and comprises **EIGHT (8)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, your Full Name, and SUSS PI Number. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a PDF or Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the PDF or Word file provided it does not exceed the file size limit of 500MB.

7. For answers which cannot be typed and require to be hand-written, ensure that question number is clearly stated on each page. All uploaded hand written answers must be clear, readable and complete. Marks will not be awarded for un-readable or incomplete images.

8. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

9. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

10. Unless otherwise specified, full details of the workings for each question are to be submitted.

Answer all questions. (Total 100 marks)

Question 1

(a) The number of operations in algorithms A and B are $2nlog_2(n)$ and n^2, respectively. Solve n_0 such that A is better than B for $n \geq n_0$. Show the procedures.

(5 marks)

(b) Analyse the time complexity of the following algorithm that computes the cumulative average of a list.

```
def cum_avg (L):
    n = len(L)
    result = [0] * n
    total = 0
    for i in range(n):
        total += L[i]
        result[i] = total / (i+1)
    return result
```

(5 marks)

Question 2

Implement an algorithm in Python to solve the problem of reversing a list, e.g., if the algorithm accepts [2,3,4,1], then it returns [1,4,3,2]. Note that the time complexity of the algorithm should be no worse than $O(n/2)$.

(10 marks)

Question 3

(a) Given a collection of medical records, we assume that each record has multiple fields, including name, age, gender, etc. Among list, tuple and string in Python, which data structure would you choose to store each record? Which data structure would you choose to store the entire collection of records? Discuss why.

(5 marks)

(b) Interpret the following code and write its output, where initialize_nested_list aims to initialize a nested list of zeros, i.e., [[0], [0], …].

```
def initialize_nested_list (n):
        L = [[0]] * n
        return L

list_1 = initialize_nested_list (5)
list_1[0][0] = 1
print (list_1)
```

(5 marks)

(c) Identify if there is any bug in the code in Question 3(b). If yes, correct the code.

(5 marks)

Question 4

(a) Given that my_q is an empty instance of the Queue ADT, the following operations are performed. Interpret the operations and write down the elements in my_q.

```
my_q.enqueue(7);
my_q.enqueue(11);
my_q.dequeue();
my_q.size();
my_q.dequeue();
my_q.enqueue(8);
my_q.enqueue(10);
```

(5 marks)

(b) If the my_q in Question 4(a) is implemented by an array-based list in Python, and the first element of my_q has the memory address of 1234. What is the memory address of the last element in my_q if each memory address occupies one byte? Justify your answer.

(5 marks)

(c) If the my_q in Question 4(a) is implemented by an array-based list in Python, what is the size of the underlying dynamic array if three more enqueue operations are performed? Justify your answer.

(5 marks)

Question 5

(a) Complete the code below to implement an algorithm in Python to remove a node in a doubly linked list called myDLL. The myDLL object has references to the head and tail of the linked list. Note that each node object has a next reference, a prev reference and an element reference.

```
def remove_node (myDLL, node):
        element = node.element
        # complete code here

        return element
```

(6 marks)

(b) Complete the code below to implement an algorithm in Python to remove a node in a singly linked list called mySLL. The mySLL object has references to the head and tail of the linked list. Note that each node object has a next reference and an element reference.

```
def remove_node (mySLL, node):
        element = node.element
        # complete code here

        return element
```

(9 marks)

Question 6

Consider the following Python implementation of the Queue ADT using a singly linked list.

```
class SLLQueue:
    class Node:
        def __init__(self, element, next):
            self.element = element
            self.next = next
    def __init__(self):
        self.head = None
        self.tail = None

    def enqueue(self, e):
        newest = self.Node(e, None)
        if self.head is None:
            self.head = newest
            self.tail = newest
        else:
            newest.next = self.head
            self.head = newest

    def dequeue(self):
        if self.head is None:
            return "Queue is empty"
        if self.head is self.tail:
            answer = self.tail.element
            self.head = self.tail = None
            return answer
        answer = self.tail.element
        node = self.head
        while node.next is not self.tail:
            node = node.next
        node.next = None
        self.tail = node
        return answer
```

(a) Analyse the time complexity of the enqueue and dequeue operations. Explain your analysis.

(5 marks)

(b) Describe the problem of the code that causes computational issues using the enqueue or dequeue operations. No code is required.

(5 marks)

Question 7

Based on the tree shown below in Figure Q7, answer the following questions.

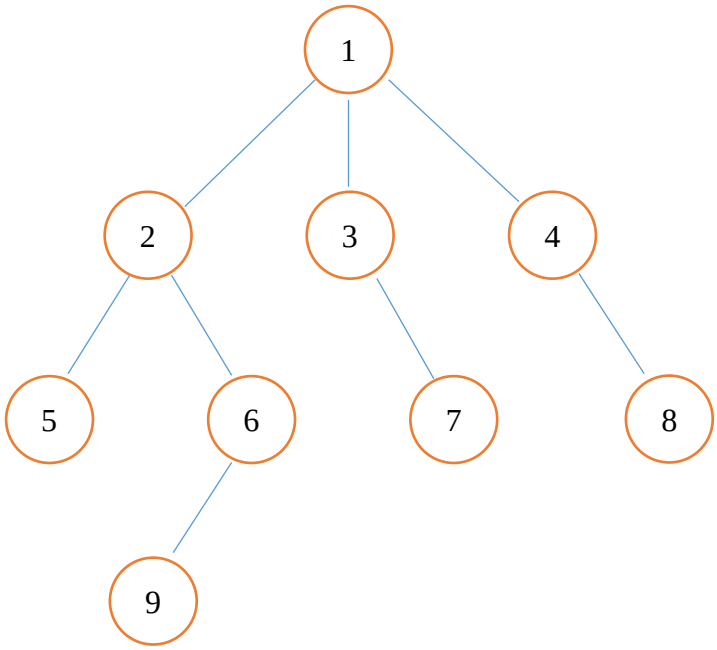

Figure Q7

(a) How many internal nodes in the tree?

(2 marks)

(b) What is the degree of node 6?

(2 marks)

(c) What is the depth of node 7?

(2 marks)

(d) What are the siblings of node 5?

(2 marks)

(e) What is the overall order of visits in preorder traversal? Analyse its time complexity.

(7 marks)

Question 8

(a) Complete the code below to implement an algorithm in Python to remove the first occurrence of a node in an array-based list called myList. Do not use built-in methods in Python list, e.g., myList.remove(node).

```
def remove_node (myList, node):
        # complete code here
        # myList.size denotes the current size of myList
        # removing an element requires decrementing the size by one

        return myList
```

(6 marks)

(b) Analyse the time complexity of the algorithm in Question 8(a). Which data structure(s) is efficient for element removal at arbitrary positions?

(4 marks)

----- END OF PAPER -----